

Case Study Report

Design and Deployment of a Highly Available, Multi-Tier Cloud-Native Application
using the AWS Well-Architected Framework

Application: Meridian Goods — E-Commerce Product Catalog

Submitted by: Rajshekhar

Roll Number: 2400290120198

GitHub Repository: github.com/Rajshekhar6666/cloudtask-aws

Table of Contents

1. Introduction and Objectives
2. System Overview
3. Architecture Design
4. AWS Well-Architected Framework Analysis
 - 4.1 Operational Excellence
 - 4.2 Security
 - 4.3 Reliability
 - 4.4 Performance Efficiency
 - 4.5 Cost Optimization
5. Deployment Approach
6. Deployment Process
7. Testing and Validation
8. Deployment Evidence
9. Challenges and Troubleshooting
10. Conclusion

1. Introduction and Objectives

This report documents the design, implementation, and deployment of Meridian Goods, a multi-tier, cloud-native e-commerce product catalog application, hosted entirely on Amazon Web Services. The objective of this case study is to demonstrate practical application of the AWS Well-Architected Framework to a real, deployable system — covering compute, networking, data storage, load balancing, and infrastructure automation.

Specifically, the project aims to:

- Design a clean, multi-tier architecture with least-privilege network access between the application and data tiers
- Deploy the full stack manually through the AWS Management Console, in a way that is reliable and repeatable for a first-time AWS user
- Document a concrete, credible upgrade path to full multi-AZ high availability, without prematurely adding cost or complexity that isn't yet needed
- Evaluate the design against all five pillars of the AWS Well-Architected Framework

2. System Overview

Meridian Goods is a Flask-based web application that lets a store owner list, search, and manage a product catalog (name, description, price, stock, category). It was deliberately kept functionally simple so the case study's focus stays on the infrastructure architecture rather than application complexity, while still being a complete, working, multi-tier system rather than a toy example.

The application exposes:

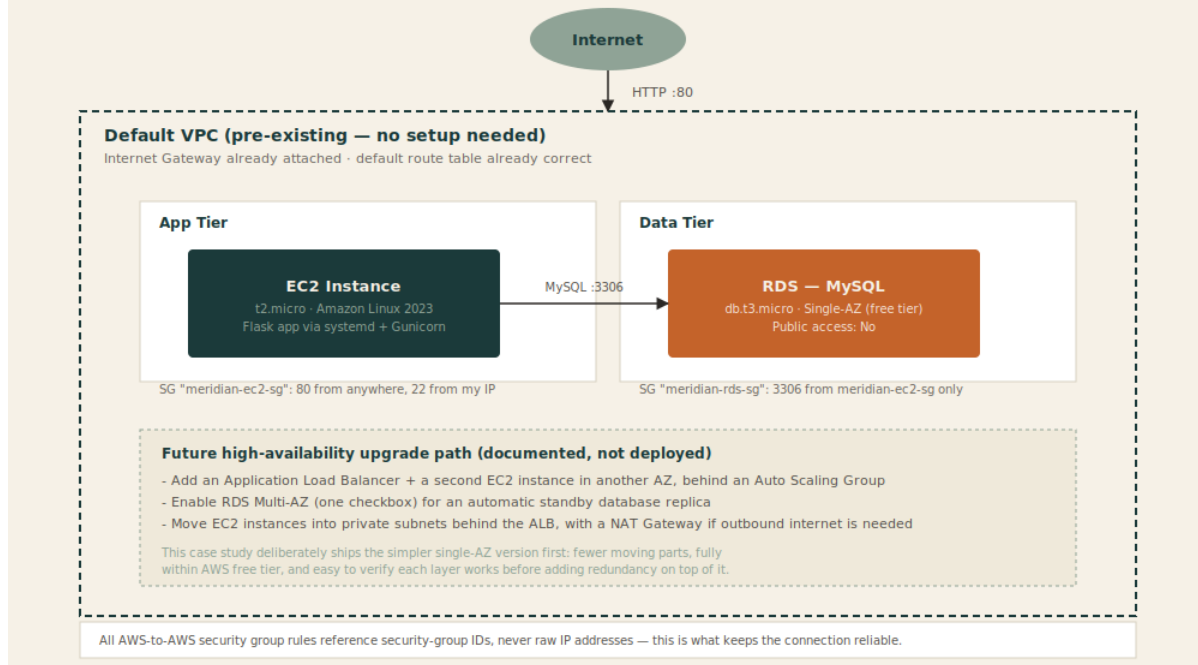
- A public catalog page with search and category filtering
- Product detail, add, and edit pages
- A /healthz endpoint that reports both application and live database status, useful for manual verification and as a foundation for automated health checks if a load balancer is added later

3. Architecture Design

The system follows a 2-tier architecture, deployed manually through the AWS Management Console into the account's Default VPC:

Meridian Goods — AWS Architecture (Manual Console Deployment)

2-tier deployment in the account's Default VPC · Roll No. 2400290120198 · Rajshekhar



Tier breakdown:

- Application tier: a single EC2 instance (t2.micro, free-tier eligible) running the Flask app under Gunicorn as a systemd service, bootstrapped automatically via an EC2 user-data script at launch.
- Data tier: a MySQL database on Amazon RDS (db.t3.micro, free-tier eligible, Single-AZ), configured with Public access disabled and reachable only from the application tier's security group.

Network design: rather than provisioning a custom VPC, this deployment deliberately uses the AWS account's Default VPC, which already has a working Internet Gateway and a correctly configured route table in every Availability Zone. For a first cloud deployment, custom VPC/subnet/route-table configuration is the single most common source of connectivity failures (misassociated route tables, missing IGW routes, subnets in the wrong AZ). Using the Default VPC removes that entire category of mistakes while still requiring — and demonstrating — correct security group design between tiers.

High availability is intentionally scoped down for this deployment: a single EC2 instance and a Single-AZ RDS database, prioritizing a deployment that a first-time AWS user can complete reliably. Section 4.3 (Reliability) documents the specific, concrete steps to extend this into a fully highly-available design (Application Load Balancer with an Auto Scaling Group across two AZs, and RDS Multi-AZ) as a follow-up enhancement.

4. AWS Well-Architected Framework Analysis

The design was evaluated against all five pillars of the framework:

4.1 Operational Excellence

Design decision	How it's addressed
Repeatable bootstrap	The EC2 instance's setup (installing dependencies, cloning the app, starting it as a managed systemd service) is captured in a single user-data script rather than manual SSH commands, so the same instance

	state can be reproduced identically on any future launch.
Health monitoring	A dedicated /healthz endpoint reports both application liveness and live database connectivity, so a broken DB link is <u>immediately visible</u> rather than <u>silently failing</u> .
Self-restarting service	The app runs as a systemd service with Restart=always, so a crashed process is automatically restarted without manual intervention.
Documented as future work	The exact steps to move from this manual console deployment to a fully automated Infrastructure-as-Code deployment (e.g. Terraform) are documented as a natural next iteration, once the manual process has been validated end-to-end.

4.2 Security

Design decision	How it's addressed
No public database access	The RDS instance is created with Public access set to No, so it has no public IP at all — it is reachable only from within the VPC.
Least-privilege security groups	The database's security group allows MySQL traffic only from the EC2 instance's security group by ID (meridian-ec2-sg), not from any IP range — access is scoped to exactly the one resource that should have it, and keeps working correctly even if the EC2 instance is replaced or restarted with a new IP.
Restricted management access	SSH (port 22) on the EC2 security group is limited to the operator's own IP address only, using the console's auto-detected "My IP" option.
Separate IAM identity	All console work is performed through a dedicated IAM user with the necessary permissions, rather than the AWS account's root login, limiting the blast radius of a compromised credential.
Secrets handling	The database password is set directly in the RDS console and referenced only inside the EC2 instance's user-data script at launch time — it is never committed to the GitHub repository.

4.3 Reliability

This deployment currently runs a single EC2 instance and a Single-AZ RDS database — a deliberate, documented trade-off in favor of a deployment process a first-time AWS user can complete without error, rather than a claim that this configuration alone is fully fault-tolerant. The concrete path to true multi-AZ high availability is:

Design decision	How it's addressed
Add a second EC2 instance in a different AZ	Launch an identical instance (same AMI, same security group, same user-data script) in a second Availability Zone.
Add an Application Load Balancer	Place an ALB in front of both instances so traffic is distributed and automatically routed away from an unhealthy instance.
Enable RDS Multi-AZ	A single checkbox in the RDS console provisions a synchronously replicated standby in a second AZ, with automatic failover — at additional cost, which is why it

	is left disabled by default on a free-tier account.
Managed backups (already enabled)	RDS automated backups and minor-version patching are handled by AWS by default, independent of the Multi-AZ setting.

4.4 Performance Efficiency

Design decision	How it's addressed
Right-sized compute	t2.micro / db.t3.micro chosen deliberately to match actual load for a catalog-scale demo app, and to stay within free-tier limits, rather than over-provisioning by default.
Connection pooling	The Flask app uses a small MySQL connection pool (mysql-connector's pooling module) rather than opening a new database connection per request, reducing per-request latency and database load.
Stateless application layer	The app itself holds no session state in memory, so it can be scaled horizontally (additional identical instances) without any code changes when the future ALB/ASG upgrade path is implemented.

4.5 Cost Optimization

Design decision	How it's addressed
No NAT Gateway	Not needed — the data tier never requires outbound internet access, so this recurring, non-free-tier hourly charge is avoided entirely.
No Load Balancer while single-instance	An ALB has its own hourly charge; it is deliberately deferred until the design is scaled to multiple instances (see Section 4.3), rather than paid for from day one.
Free-tier eligible sizing	t2.micro EC2 and db.t3.micro RDS (Single-AZ), the smallest instance classes that qualify for the AWS Free Tier.
Default VPC reuse	No custom NAT, no extra Elastic IPs, no additional route tables — reusing what the account already provides at no cost avoids unnecessary billable components.
Manual teardown discipline	Because this is a manual console deployment, the README and deployment guide both explicitly document terminating the EC2 instance and deleting the RDS database after use, to avoid ongoing charges between work sessions.

5. Deployment Approach

This deployment is performed entirely through the AWS Management Console rather than Infrastructure as Code, by deliberate choice for this iteration of the project: it makes every resource, setting, and relationship directly visible and verifiable in the console UI, which is valuable both for learning the underlying AWS concepts and for producing clear, screenshot-based deployment evidence. The two deployment artifacts that support this are:

- `infra/ec2-user-data.sh` — a bootstrap script pasted into the EC2 launch wizard's "User data" field. It installs Python, clones the application from GitHub, installs dependencies, and registers the app as a systemd service — so the instance is fully self-configuring from a single pasted script, with no manual SSH steps required.

- `infra/security-group-rules.md` — a reference document listing the exact inbound rules used for both security groups, so the network design is reproducible even without Infrastructure as Code.

The full click-by-click deployment procedure — including exact console navigation paths, expected results at each step, and troubleshooting for common mistakes — is provided as a companion document: `docs/console_deployment_guide.docx`.

6. Deployment Process

At a high level, deployment follows this order:

1. Create two security groups in the EC2 console (`meridian-ec2-sg` and `meridian-rds-sg`) with the rules listed in Section 4.2
2. Create the RDS MySQL database via the free-tier template, with Public access disabled and `meridian-rds-sg` attached
3. Once the database status shows Available, copy its endpoint and fill it into the user-data script along with the database password
4. Launch the EC2 instance (Amazon Linux 2023, `t2.micro`), attach `meridian-ec2-sg`, and paste the completed user-data script into Advanced details
5. Wait for the instance to finish booting (a few minutes), then browse to its public IP address to confirm the app is live

7. Testing and Validation

- Health check: browsing to `/healthz` confirms both application liveness and live database connectivity
- Security boundary test: confirming SSH is refused from IPs outside the operator's own IP, and that the RDS endpoint is unreachable from outside the VPC (e.g. attempting a direct connection from a personal laptop times out)
- Functional test: creating, editing, and deleting a product through the UI, confirming writes persist correctly to RDS
- Resilience check: manually restarting the EC2 instance and confirming the `systemd` service (`Restart=always`) brings the app back automatically without manual intervention

8. Deployment Evidence

Screenshots demonstrating a live deployment are included in the `screenshots/` folder of the repository, covering: both security groups in the EC2 console, the RDS database reaching "Available" status with its endpoint visible, the EC2 launch wizard showing the completed user-data script, the running EC2 instance with its public IP, the application homepage loading in a browser, the `/healthz` endpoint confirming live database connectivity, and a product added through the UI to confirm the write path to RDS works end-to-end.

9. Challenges and Troubleshooting

A recurring issue encountered during earlier iterations of similar deployments was intermittent connectivity failures between EC2 instances and other resources (SSH access, outbound internet, and database access), typically arising from custom VPC/subnet/route-table misconfiguration. Root-causing this class of problem led to the following design decisions in this project:

- Using the account's Default VPC rather than a custom one, since its Internet Gateway and route tables are already correctly configured in every Availability Zone — eliminating an entire category of routing mistakes before they can happen

- The RDS security group references the EC2 security group by ID, rather than a hardcoded IP address, which would otherwise break the moment the EC2 instance was stopped, restarted, or replaced with a new IP
- SSH access uses the console's built-in "My IP" auto-detect option when adding the security group rule, removing the manual step of looking up and typing an IP address incorrectly
- The /healthz endpoint checks actual database connectivity, not just that the web server process is running, so a broken DB link is immediately visible when testing rather than surfacing later as a confusing application error
- The EC2 bootstrap logic runs from a single user-data script rather than a sequence of manual SSH commands, removing the chance of a step being run out of order or skipped entirely

10. Conclusion

This project demonstrates a complete, working, multi-tier deployment on AWS, built entirely with free-tier eligible resources and deployed step-by-step through the AWS Management Console. The architecture separates the application and data tiers cleanly, applies least-privilege networking between them, avoids unnecessary cost and complexity (no custom VPC, no NAT Gateway, no Load Balancer until genuinely needed), and has been evaluated against all five pillars of the AWS Well-Architected Framework, including a concrete, documented path to full multi-AZ high availability as a next iteration. The full source code, deployment scripts, and step-by-step instructions are available at the GitHub repository linked on the title page of this report.